

# PRD: Recall-AI Modernization & Extension Migration

## 1. Executive Summary

The objective is to modernize **Recall-AI**—a spaced-repetition vocabulary trainer for ESL learners—by migrating its frontend from **HTMX/Jinja2** to a **React/TypeScript SPA** and then expanding it into a **Voice-First Chrome Extension**. This transition will leverage **Tanstack (Query, Router, Table)** and demonstrate **Speechify-level engineering standards** in performance, auditory learning, and scalable architecture.

## 2. Phase 0: The Great Migration (HTMX -> React)

Before building the extension, the core web application must be modernized to provide the API-first foundation required for a Chrome Extension.

### 2.1 Frontend Modernization

- **Framework:** React 19 + TypeScript.
- **Routing:** **Tanstack Router** for type-safe routing and state-driven navigation.
- **Data Fetching:** **Tanstack Query** to replace HTMX's server-side partial rendering.
- **State Management:** **Zustand** for lightweight local state (e.g., current review session).
- **Styling:** Retain TailwindCSS, but migrate from global CSS to a component-driven architecture.

### 2.2 Backend Adaptation (FastAPI)

- **API-First Transition:** Convert Jinja2 template routes into **Pydantic-powered JSON endpoints**.
- **Deployment:** Maintain the current **Railway** setup, but update the `railway.json` to handle the decoupled frontend/backend build process (e.g., using Railway adapters for static site hosting).

---

## 3. Phase 1: The Voice-First Chrome Extension

Once the React foundation is solid, the product will be ported to a **Manifest V3 Extension**.

### 3.1 Architecture: Manifest V3

- **Service Worker ( `background.js` )**: The ephemeral task runner managing the SM-2 review session state.
- **Offscreen Document ( `offscreen.html` )**: The "secret tunnel" for **Neural TTS playback** of vocabulary and LLM-generated example sentences.
- **Performance: React Virtualization ( `react-window` )** for the vocabulary archive view to ensure 60FPS scrolling for thousands of items.

### 3.2 Scaling Strategy: Firebase / Postgres

- **Denormalization**: Use a "Daily Batch" document pattern to minimize database reads during the review session.
  - **Sync**: Real-time synchronization of SM-2 ratings (Easy, Good, Hard) back to the FastAPI backend.
- 

## 4. Technical Requirements (For AI Coding Agents)

### 4.1 Prerequisite: Frontend Refactor

- ☐ **Decouple**: Remove all Jinja2 templates and HTMX attributes.
- ☐ **Type Safety**: Generate TypeScript interfaces from Pydantic schemas to ensure end-to-end type safety.
- ☐ **API Client**: Build a centralized API client using Tanstack Query for all CRUD operations.
- ☐ **Clean Sweep (Post-Parity)**: Once 1:1 functional parity is confirmed, **purge all unused code**:
  - Remove all `.html` templates from `/templates`.
  - Delete `htmx` and `jinja2` from `package.json` and Python dependencies.
  - Remove legacy Jinja2-based routes from the FastAPI backend.
  - Clean up any unused Tailwind classes or CSS partials only used by the legacy UI.

### 4.2 Extension Implementation

- ☐ **Manifest V3**: Setup `manifest.json` with `offscreen` and `identity` permissions.
- ☐ **Background Sync**: Implement the Service Worker to fetch the daily review batch from the FastAPI backend.
- ☐ **Voice Engine**: Setup the Offscreen Documents API to read cards aloud automatically

upon flip.

---

## 5. Definition of Done (DoD)

- ☐ **Migration:** The web app is a fully functional React SPA with no remaining HTMX/Jinja2 code.
  - ☐ **Voice-First:** Every vocabulary card in the extension is read aloud using the Offscreen API.
  - ☐ **Performance:** 60FPS scrolling in the vocabulary archive; initial review load < 500ms.
  - ☐ **Sync:** Completing a review in the extension perfectly updates the SM-2 curve in the backend.
- 

## 6. Speechify Interview Talking Points (The "Modernization" Narrative)

- *"I recognized that a server-rendered HTMX architecture was a bottleneck for the voice-first, low-latency experience I wanted to build. I spearheaded a migration to a React/TypeScript SPA using Tanstack Query and Router to create a solid API-first foundation."*
- *"I treated the Chrome Extension as a first-class citizen, using Manifest V3 and the Offscreen Documents API to implement auditory learning—directly mirroring Speechify's core value of removing reading barriers."*
- *"I optimized the SM-2 sync by moving from server-side partials to a batch-loading strategy, ensuring that the 'forgetting curve' remains accurate even in a distributed extension/web environment."*
- *"I used react-window to virtualize the vocabulary archive, proving that I can maintain 60FPS performance for power users with thousands of cards."*